

Отчёт по лабораторной работе №8

по дисциплине «Архитектура компьютера»

Дембирел Тимур Леонидович

Российский университет дружбы народов
Факультет физико-математических и естественных наук
Кафедра прикладной информатики и теории вероятностей
Москва 2025г.

Содержание

1	Цель работы	6
2	Выполнение лабораторной работы	7
3	Вывод	16

Список иллюстраций

2.1	Создаем	8
2.2	Вводим	8
2.3	Проверяем	9
2.4	Изменяем	9
2.5	Проверяем	10
2.6	Вносим изменения	10
2.7	Проверяем	11
2.8	Создаем и вводим	11
2.9	Проверяем	12
2.10	Создаем и вводим	12
2.11	Запускаем	13
2.12	Изменяем	13
2.13	Проверяем	14
2.14	Текст программы	15
2.15	Проверка	15

Список таблиц

2.1	Основные команды, используемые при работе	7
-----	---	---

1 Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2 Выполнение лабораторной работы

В Таблица 2.1 приведены основные команды, используемые при работе с git, элементами программирования и компиляцией программ.

Таблица 2.1: Основные команды, используемые при работе

Команды	Описание
<code>touch</code>	Создание файла
<code>nasm</code>	Компилятор ассемблера NASM
<code>git add</code>	Добавление изменений в индекс
<code>git commit</code>	Создание коммита
<code>git push</code>	Отправка изменений на удаленный репозиторий
<code>ld</code>	Связывает объектные файлы в исполняемую программу
<code>mkdir</code>	Создает новую директорию
<code>push</code>	Размещает значение в стеке
<code>pop</code>	Извлекает значение из стека
<code>loop</code>	Уменьшает регистр ECX на 1, переходит на метку, если ECX $\neq$ 0

Создаем каталог для программ лабораторной работы № 8, перейдем в него и со-

создаем файл lab8-1.asm рис. 2.1.

```
timur@timur-VirtualBox:~$ mkdir ~/work/arch-pc/lab08
timur@timur-VirtualBox:~$ cd ~/work/arch-pc/lab08
timur@timur-VirtualBox:~/work/arch-pc/lab08$ touch lab8-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab08$
```

Рисунок 2.1: Создаем

Вводим в файл lab8-1.asm текст программы из листинга 8.1 рис. 2.2.

```
GNU nano 7.2 lab8-1.asm
#include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
mov [N],ecx
mov eax,[N]
call iprintLF ; Вывод значения `N`
loop label ; `ecx=ecx-1` и если `ecx` не '0'
; переход на `label`
call quit
```

Рисунок 2.2: Вводим

Создаем исполняемый файл и проверяем его рис. 2.3.


```

timur@timur-VirtualBox:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 12
12
11
10
9
8
7
6
5
4
3
2
1
timur@timur-VirtualBox:~/work/arch-pc/lab08$ █

```

Рисунок 2.3: Проверяем

Изменим текст программы добавив изменение значение регистра ecx в цикле рис. 2.4.

```

Label:
sub ecx,1 ; `ecx=ecx-1`
mov [N],ecx
mov eax,[N]
call iprintLF
loop label
call quit

```

Рисунок 2.4: Изменяем

И также создаем исполняемый файл и проверяем его работу рис. 2.5.

```

timur@timur-VirtualBox:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 12
11
9
7
5
3
1
timur@timur-VirtualBox:~/work/arch-pc/lab08$

```

Рисунок 2.5: Проверяем

Регистр каждый раз уменьшается на 2, а число проходов не соответствует.

Для использования регистра `ecx` в цикле и сохранения корректности работы программы используем стек. Внесем изменения в текст программы добавив команды `push` и `pop` для сохранения значения счетчика цикла `loop` рис. 2.6.

```

label:
push ecx ; добавление значения ecx в стек
sub ecx,1
mov [N],ecx
mov eax,[N]
call iprintLF
pop ecx ; извлечение значения ecx из стека
loop label
call quit

```

Рисунок 2.6: Вносим изменения

Создаем исполняемый файл и проверяем рис. 2.7.

```

timur@timur-VirtualBox:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 12
11
10
9
8
7
6
5
4
3
2
1
0
timur@timur-VirtualBox:~/work/arch-pc/lab08$

```

Рисунок 2.7: Проверяем

Число проходов цикла соответствует значению N.

Создаем файл lab8-2.asm в каталоге ~/work/arch-pc/lab08 и вводим в него текст программы из листинга 8.2 рис. 2.8.

```

GNU nano 7.2 lab8-2.asm *
#include 'in_out.asm'
SECTION .text
global _start
_start:
пор есх ; Извлекаем из стека в `есх` количество
; аргументов (первое значение в стеке)
пор edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub есх, 1 ; Уменьшаем `есх` на 1 (количество
; аргументов без названия программы)
next:
cmp есх, 0 ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
пор еах ; иначе извлекаем аргумент из стека
call sprintLF ; вызываем функцию печати
loop next ; переход к обработке следующего
; аргумента (переход на метку `next`)
_end:
call quit

```

Рисунок 2.8: Создаем и вводим

Создаем исполняемый файл и запускаем его, указав аргументы рис. 2.9.

```
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ./lab8-2 аргумент1 аргумент 2 'аргумент 3'
аргумент1
аргумент
2
аргумент 3
timur@timur-VirtualBox:~/work/arch-pc/lab08$
```

Рисунок 2.9: Проверяем

Программа обработала все аргументы.

Создим файл lab8-3.asm в каталоге ~/work/archpc/lab08 и введем в него текст программы из листинга 8.3 рис. 2.10.

```
GNU nano 7.2 lab8-3.asm *
#include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
    ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
    ; (второе значение в стеке)
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
    ; аргументов без названия программы)
    mov esi, 0 ; Используем `esi` для хранения
    ; промежуточных сумм
next:
    cmp ecx,0h ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет выходим из цикла
    ; (переход на метку `_end`)
    pop eax ; иначе извлекаем следующий аргумент из стека
    call atoi ; преобразуем символ в число
    add esi,eax ; добавляем к промежуточной сумме
    ; след. аргумент `esi=esi+eax`
    loop next ; переход к обработке следующего аргумента
_end:
    mov eax, msg ; вывод сообщения "Результат: "
    call sprint
    mov eax, esi ; записываем сумму в регистр `eax`
    call iprintLF ; печать результата
```

Рисунок 2.10: Создаем и вводим

Создаем исполняемый файл и запускаем его, указав аргументы рис. 2.11.

```

timur@timur-VirtualBox:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 47
timur@timur-VirtualBox:~/work/arch-pc/lab08$

```

Рисунок 2.11: Запускаем

Изменим текст программы из листинга 8.3 для вычисления произведения аргументов командной строки рис. 2.12.

```

GNU nano 7.2 lab8-3.asm *
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
пор есх ; Извлекаем из стека в `есх` количество
; аргументов (первое значение в стеке)
пор edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub есх,1 ; Уменьшаем `есх` на 1 (количество
; аргументов без названия программы)
mov esi, 1 ; Используем `esi` для хранения
; промежуточного произведения (НАЧИНАЕМ С 1)
next:
cmp есх,0h ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
пор еах ; иначе извлекаем следующий аргумент из стека
call atoi ; преобразуем символ в число
imul esi,еах ; УМНОЖАЕМ промежуточное произведение
; на след. аргумент `esi=esi*еах`
loop next ; переход к обработке следующего аргумента
_end:
mov еах, msg ; вывод сообщения "Результат: "
call sprint
mov еах, esi ; записываем произведение в регистр `еах`
call iprintLF ; печать результата
call quit ; завершение программы

```

Рисунок 2.12: Изменяем

Создаем исполняемый файл и проверяем рис. 2.13.

```

timur@timur-VirtualBox:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 54600
timur@timur-VirtualBox:~/work/arch-pc/lab08$

```

Рисунок 2.13: Проверяем

#Задание для самостоятельной работы

1. Напишите программу, которая находит сумму значений функции $f(x)$ для $x = x_1, x_2, \dots, x_n$, т.е. программа должна выводить значение $f(x) + f(x_2) + \dots + f(x_n)$. Значения x_i передаются как аргументы. Вид функции $f(x)$ выбрать из таблицы 8.1 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу на нескольких наборах $x = x_1, x_2, \dots, x_n$. Пример работы программы для функции $f(x) = x + 2$ и набора $x_1 = 1, x_2 = 2, x_3 = 3, x_4 = 4$:
user@dk4n31:~\$./main 1 2 3 4 Функция: $f(x)=x+2$ Результат: 18 user@dk4n31:~\$
рис. 2.14 рис. 2.15

```

GNU nano 7.2                                     tasklab8.asm *
%include 'in_out.asm'
SECTION .data
msg_func db "Функция: f(x)=10x-5",0
msg_result db "Результат: ",0
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем количество аргументов
    pop edx ; Извлекаем имя программы
    sub ecx,1 ; Уменьшаем количество аргументов
    mov esi, 0 ; Используем esi для хранения суммы

    ; Выводим информацию о функции
    mov eax, msg_func
    call sprintLF

next:
    cmp ecx,0h ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет - выходим
    pop eax ; извлекаем аргумент из стека
    call atoi ; преобразуем в число

    ; Вычисляем f(x) = 10x - 5
    mov ebx, 10
    imul eax, ebx
    sub eax, 5

    add esi,eax ; добавляем к сумме

```

Рисунок 2.14: Текст программы

```

timur@timur-VirtualBox:~/work/arch-pc/lab08$ nasm -f elf tasklab8.asm
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 -o tasklab8 tasklab8.o
timur@timur-VirtualBox:~/work/arch-pc/lab08$ ./tasklab8 1 2 3 4
Функция: f(x)=10x-5
Результат: 80
timur@timur-VirtualBox:~/work/arch-pc/lab08$

```

Рисунок 2.15: Проверка

3 Вывод

В ходе данной лабораторной работы я изучил команды условных и безусловных переходов. Приобрел навыки написания программ с использованием переходов. Познакомился с назначением и структурой файлов листинга.